





Packet Sniffer

Cairo University

Faculty of Computers and Artificial Intelligence

Department of Information Technology

Department of Computer Science

PACKET SNIFFER

Graduation Project Report

Abdelhadi Mohammed-Salih
IT Department

Abubakr Khalid
IT Department

Al-Monzir Ali
IT Department

Braa Abdelhafeiz
IT Department

Mazin Mohammed-Alhassan
CS Department

Supervised By: *Dr.Reda Abdelwahab & Dr.Khalid Wassif*

July 3, 2024

Abstract

This abstract outlines a research project focused on the develop- ment and analysis of a packet sniffing tool. The purpose is to con- tribute to the field of network security by developing a tool that en- hances the understanding of data transmission. The project is motivated by the growing complexity of network communication and the need for effective tools to dissect this intricate process.

The implemented software features include efficient packet capturing and detailed protocol examination. Advanced methods are employed for packet capturing and detailed protocol analysis. The software features robust capabilities to examine the nuances of data transfer. Conclusions drawn from the study underscore the tool's efficacy in providing insights into network behavior. The significance of this research lies in advancing our comprehension of data flow within networks, offering a valuable resource for network administrators and analysts.

This project addresses fundamental questions about data transmission, its importance in contemporary networking, the methodology used for analysis, and the meaningful insights gained from the conclusions.

Acknowledgment

After our work, we would like to thank both Dr. Reda Abdelwahab And Dr. Khalid Wassif for allowing us to work on the graduation project under Their supervision. They also supported from the beginning by presenting to that idea and the technology used in it, not leaving, and always directing us to reach the next step in research and correct our endeavor path during the project. We thank Them for Their time with us to advise us and listen to the problems we face and try to find solutions to them. And of course, we don't forget to thank our families, especially our parents for their unconditional support mentally, financially, encouragement, patience, and help over the years. We are forever indebted to our parents, who always kept us in their prayers and pushed us to success. We also thank our friends and colleagues for their support and feedback. Finally, for our faculty for providing a suitable environment that led us to represent the best image that computer science graduates of Cairo University are supposed to represent.

Contents

Abstract 1

Acknowledgement 2

Chapter 1: 7

1.1 Background and Motivation 8

1.2 Problem Statement 8

1.3 Objectives 8

1.4 Project Scope and Limitations 9

1.5 Project Methodology 11

1.6 Project Report Outline 15

Chapter 2: 16

2.1 Tcpdump 17

2.2 Wireshark 18

2.3 Tcpdump vs. Wireshark 18

2.3.1 Options 18

2.3.2 Memory Usage 19

2.3.3 Other Resources 19

Chapter 3: **20**

3.1 General Overview 21

3.2 Process Flow 22

3.2.1 Functional Requirements 22

3.2.1 Non Functional Requirements 23

Chapter 4: **25**

4.1 Overview 26

4.1.1 Key Components 26

4.2 Functional Flow 27

Chapter 5: **34**

5.1 Technologies and Libraries 35

5.2 GUI Implementation 38

5.3 GUI Design 39

Chapter 6: **45**

6.1 Conclusion 46

6.1.1 Key Achievements 46

6.2 Recommendations For Future Work 46

6.3 Final Thoughts 47

References 48

List of Figures

1	Use Case Diagram	24
2	Flow Diagram of The Packet Sniffer	29
3	Class Diagram of The Packet Sniffer	30
4	Sequence Diagram (1)	31
5	Sequence Diagram (2)	33



Packet Sniffer

Chapter 1: Introduction

1.1 Background and Motivation

The evolution of network analysis tools has been marked by the increasing complexity of data transmission. The motivation behind developing a packet sniffing tool stems from the growing need to address the intricate nature of modern network communication and to provide a more advanced solution for efficient packet analysis.

1.2 Problem Statement

Contemporary network monitoring faces challenges in effectively capturing and understanding data flows. The packet sniffing tool aims to tackle these challenges by focusing on improving packet analysis methods, thereby enhancing the overall comprehension of network behavior.

1.3 Objectives

The objectives of the packet sniffing tool project include:

- Improving the efficiency of packet capturing.
- Enhancing the detailed analysis of network protocols.
- Contributing to a deeper understanding of data transmission within networks.

1.4 Project Scope and Limitations

The project focuses on:

Project Scope:

- Objective**:** Develop a robust and user-friendly packet sniffer application designed to monitor, capture, and analyze network traffic in real-time for medium to large-scale networks.
- Target Networks: This application is optimized for medium to large-scale network environments, ensuring efficient performance and accurate data handling.
- Protocol Analysis: The application provides comprehensive analysis of key network protocols including Ethernet, IPv4, IPv6, TCP, UDP, DNS, HTTP, and SSL. It categorizes and counts packets for each protocol, facilitating detailed traffic insights.
- User Interface: The GUI, built using ImGui, offers an intuitive and interactive experience. Users can:
    - Select Network Devices: Choose from a list of available network devices to monitor.
    - View Real-Time Statistics: Display real-time statistics of captured packets, including protocol-specific counts.
    - Inspect Packets: Examine detailed information of individual packets, including their protocol layers and raw data.
    - Memory Editing: Use the integrated memory editor for advanced inspection of packet data.
- Cross-Platform Compatibility: The application supports both Windows and Unix-like systems, with conditional compilation to ensure seamless operation across different operating systems.

Limitations:

- Supported Platforms: Although the application supports both Windows and Unix-like systems, certain functionalities and optimizations are primarily designed for Windows.
- Device Compatibility: The packet sniffer relies on the PcapPlusPlus library for packet capture, thus compatibility is limited to network devices supported by this library.
- Protocol Filtering: The current implementation filters packets based on the "ip" keyword, which may limit more granular protocol filtering options. Future enhancements could include more sophisticated filtering capabilities.
- Visualization Scope: The application focuses on providing real-time statistics and detailed packet inspection but does not include advanced features such as historical data analysis or complex network topology mapping.
- Performance Considerations: While optimized for medium to large-scale networks, extremely high traffic volumes might impact performance and responsiveness. Further optimizations may be necessary for handling exceptionally high loads.

This comprehensive scope outlines the intended functionality, user experience, and constraints of the packet sniffer application, ensuring it meets the needs of network administrators and analysts in medium to large-scale network environments.

1.5 Project Methodology

The development of the packet sniffing tool adheres to a structured and comprehensive methodology, ensuring the creation of a robust and effective network analysis application. The methodology comprises the following phases:

1. **Research and Analysis:**
    - Literature Review: Conduct an extensive review of existing packet sniffing tools and network analysis techniques to identify best practices and gaps in current solutions.
    - Technology Evaluation: Assess various libraries and frameworks for packet capture and analysis, leading to the selection of PcapPlusPlus for its flexibility and performance.
    - Use Case Identification: Define key use cases and user requirements by consulting network administrators and IT professionals to ensure the tool addresses real-world needs.

2. **Design:**

   - System Architecture: Develop a modular architecture that separates packet capture, protocol analysis, and user interface components to enhance maintainability and scalability.
   - User Interface Design: Create a user-friendly GUI using ImGui, ensuring intuitive navigation and interaction for network administrators.
   - Cross-Platform Compatibility: Design the application with conditional compilation directives to support both Windows and Unix-like systems, leveraging platform-specific optimizations where necessary.

3. **Implementation:**

   - Packet Capture Mechanisms: Implement efficient packet capture mechanisms using PcapPlusPlus. Develop functions to open network devices, apply filters, and handle packet capture callbacks.
   - Protocol Analysis: Develop detailed protocol analysis capabilities. Implement methods to classify and count packets by protocol type, leveraging PcapPlusPlus's protocol detection features.
   - Concurrency Management: Use mutexes to ensure thread-safe access to shared resources such as captured packet data and statistics, maintaining application stability and performance.
   - Real-Time Statistics: Implement real-time statistics display in the GUI, showing protocol-specific packet counts and other relevant metrics.

4. **User Interface Development:**
    - Device Selection: Create an interface for selecting and managing network devices, allowing users to start and stop packet capture on chosen devices.
    - Packet Display: Implement a detailed packet list view with color-coded protocol layers and selectable packets for inspection.
    - Packet Inspection: Develop a packet inspector view, providing a detailed breakdown of each packet's protocol layers and raw data, supported by an integrated memory editor for advanced analysis.

5. **Testing and Refinement:**
    - Unit Testing: Write unit tests for individual components, ensuring correctness and reliability of packet capture and analysis functions.
    - Integration Testing: Perform integration testing to validate the interaction between different components, ensuring smooth data flow from packet capture to GUI display.
    - User Testing: Conduct user testing sessions with network administrators to gather feedback on usability, functionality, and performance, making iterative improvements based on user insights.
    - Performance Optimization: Analyze and optimize the application's performance, focusing on minimizing latency and maximizing throughput, especially in high-traffic network environments.

6. **Deployment and Documentation:**

   - Deployment: Prepare the application for deployment on both Windows and Unix-like systems, providing clear installation instructions.
   - Documentation: Develop comprehensive user documentation, including usage guides, troubleshooting tips, and API references for developers extending the tool.

This detailed methodology ensures a systematic approach to developing a robust and efficient packet sniffing tool, addressing both functional and non-functional requirements through rigorous research, design, implementation, testing, and refinement.

1.6 Project Report Outline

The report is structured as follows:

- Section 2: Literature Review
- Section 3: Design and Implementation
- Section 4: Testing and Validation
- Section 5: Results and Analysis
- Section 6: Conclusion and Future Work.



Packet Sniffer

Chapter 2: Related Work

There are various implementations of packet sniffing tools considering the development and popularization of network technology. And the management, maintenance and monitoring of networks is important to keep them smooth and improve economic efficiency.

A lot of research has been conducted by experts on this topic. So, we first tried to analyze these existing works to find out the best possible ways to implement our tool. In our research, we found that this topic can be implemented with various libraries which are all wrapped around libpcap. We then decided to use a specific one that is suitable for our use.

There are hundreds of packet sniffers present in the market accommodating to different industry and personal requirements, but of all these, Tcp- dump and Wireshark are the most widely used and liked tools.

2.1 Tcpdump

Tcpdump is highlighted as an excellent and lightweight network traffic capturing tool, particularly beneficial for network learners and enthusiasts aim- ing to comprehend TCP/IP thoroughly.

Tcpdump operates by dumping packets in raw format, allowing users to analyze the data without extensive pre-processing. To enhance usability, Tcpdump offers various options such as displaying human-readable timestamps (’-t’), showing Hex and ASCII packet contents (’-X’), and increasing packet information (’-v’, ’-vv’, ’-vvv’). This approach facilitates clearer in- sights into networking and packet concepts by providing flexibility in how captured packet details are presented and analyzed.

2.2 Wireshark

Wireshark stands out as the most widely used network sniffing tool, credited for its user-friendly graphical interface and robust capturing and filtering capabilities. Similar to Tcpdump, Wireshark is freely available. Its versatility is highlighted by its capacity to scan various networks, including Ethernets, Wi-Fi, monitor mode, and Bluetooth. Wireshark's global adoption by indus- try leaders extends beyond mere sniffing; it functions as an Intrusion Detec- tion System (IDS), promptly identifying and reporting threats like Denial of Service (DOS) attacks, firewall breaches, and other security incidents. Wire- shark's capability to delve into packet details at the bit level, along with its color-coded protocol highlighting, enhances its effectiveness in network analysis and security monitoring.

2.3 Tcpdump vs. Wireshark

2.3.1 Options

- Tcpdump offers selective packet dumping based on matching strings, numbers, or C program fragments with options like -dd or -ddd. In contrast, Wireshark dumps all packets and applies matching filters afterward.

- Wireshark supports dumping packets in multiple files with a single command (-b) and creating a ring buffer, an option that is unavailable in Tcpdump.

- Wireshark can collect and display real-time statistics through the -z command, a feature absent in Tcpdump

2.3.2 Memory Usage

- After multiple analyses using the Htop utility on Ubuntu in Ethernet mode (eth0), Tcpdump's VIRT (Virtual Size) is around 27-29 Mb, while Wireshark's is significantly higher at 620-640 Mb.
- The SHR (Shared Memory) for Tcpdump is 5.1-5.3 Mb, and for Wire- shark, it is 35-36 Mb.
- The MEM% (Memory Percentage) for Tcpdump is 0.6%, while Wire- shark has a higher share at 8.8-9.6%. In conclusion, Tcpdump uses sig- nificantly less memory than Wireshark, making it the preferable choice in this aspect.

2.3.3 Other Resources

- In Power top analysis, Tcpdump is more efficient than Wireshark, using 37-38 microseconds per second compared to Wireshark's 1.5 microsec- onds.
- Tcpdump has 1 wakeup/second, while Wireshark has 14, and Wire- shark's graphics wakeups are 0.1, not applicable in Tcpdump.
- Tcpdump's battery usage is negligible at 0%, whereas Wireshark uses 0.2%. Despite increased monitor mode usage, Tcpdump excels in power and processor efficiency over Wireshark.



Packet Sniffer

Chapter 3: System Analysis

A GUI Application to analyze the network packets with advanced packet filtering capabilities focusing on small memory footprint, minimal resource usage and, real-time analysis.

3.1 General Overview

To achieve this the following design is followed as shown in Figure 1 the system starts by the following steps:

- Initializing the logging system, then creating a window after that initializing the graphics context (OpenGL).
- Creating another thread for capturing packets and kernel communication, doing this on the main thread results in a bad experience as we are rendering the UI and responding to the OS events on it.
- Retrieving the available network interfaces on the system from the kernel.
- Processing the GUI frame, displaying the list of devices to the user and waiting on a device selection.
- After Selecting a device the application hooks-up to the kernel and receives every packet that arrives to the network interface.
- If there is an active device the GUI frame displays each packet and allows the user to choose a packet for further inspection.

3.2 Process Flow

3.2.1 Functional Requirements

- Device Selection:

The system list all available network devices.
The user is able to select a network device for packet capture.

- Packet Capture:

The system captures packets from the selected network device in real-time.
The system uses a callback function to process each captured packet.

- Packet Processing:

The system identifies and categorize packets into different types (Ethernet, IPv4, IPv6, TCP, UDP, DNS, HTTP, SSL).
The system extracts and display header information for each packet type.
The system extracts and display payload data for each packet.

- 3.2.1.4 Metrics Recording:

The system records latency for each packet.
The system detects and record packet loss based on sequence numbers.
The system records bandwidth usage in bytes per second for each flow.
The system detects bandwidth anomalies based on a predefined threshold.

- Statistics Display:

The system maintains and display counts of different packet types.
The system updates the statistics in real-time as packets are captured.

- User Interface:

The system must provide a graphical user interface (GUI) to interact with the user.
The GUI displays a main menu with options to close the application and select a capture device.
The GUI displays a list of captured packets and their types.
The GUI allows the user to select a packet from the list to view detailed information.
The GUI displays header and payload information of the selected packet.

- Logging:

The system logs performance metrics, including latency, packet loss, and bandwidth usage.
The system logs detected anomalies.

3.2.2 Non-Functional Requirements

- Performance:

The system processes packets in real-time without significant delay.
The GUI updates packet statistics and details with minimal latency.

- Scalability:

The system handles high volumes of network traffic efficiently.
The system is able to manage and display large numbers of captured packets without performance degradation.

- Reliability:

The system accurately capture and process packets.
The system reliably record and report performance metrics.

- Usability:

The GUI is intuitive and easy to navigate.
The system provides clear and concise information about captured packets and metrics.
The system allows easy selection and switching of network devices.

- Portability:

The system is compatible with multiple operating systems (e.g., Windows, Linux).
The system handles platform-specific differences (e.g., device naming conventions).

- Maintainability:

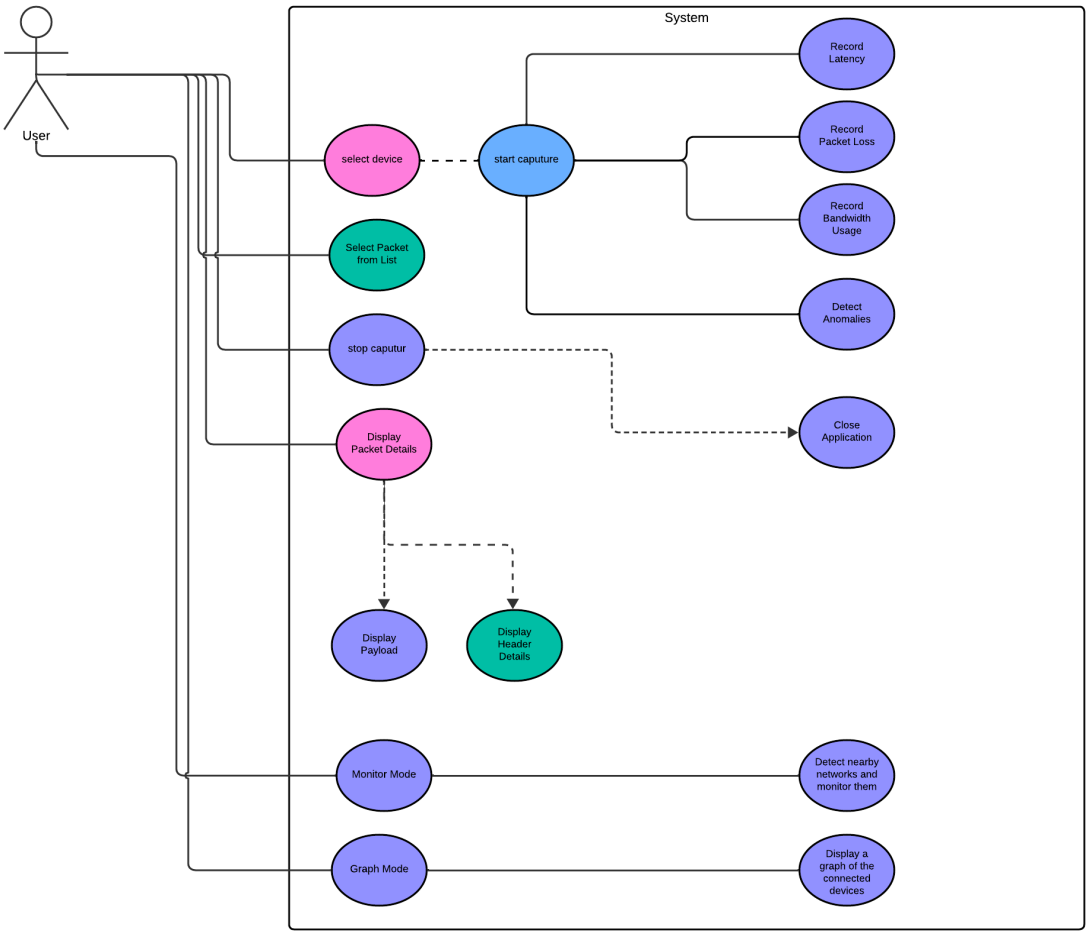
The system is structured in a modular way to facilitate easy maintenance and updates.
The code is well-documented to assist developers in understanding and modifying it.

- Security:

The system ensures secure handling of captured packet data.
The system does not expose sensitive information in logs or through the GUI.

- Resource Usage:

The system efficiently manages memory and CPU usage to avoid excessive resource consumption.
The system provides options to clear or reset statistics to manage long-term resource usage.



Use Case Diagram



Packet Sniffer

Chapter 4: System Analysis

4.1 Overview

This code is a packet sniffer application. It captures and analyzes network packets in real-time, extracting useful information from them, and displaying this information in a graphical user interface (GUI). Additionally, it records performance metrics such as latency, packet loss, and bandwidth usage, and can detect anomalies.

4.1.1 Key Components

1. **Header Files and Libraries:**

   The code includes various libraries and headers for packet processing (`Packet.h`, `ProtocolType.h`, etc.), threading (`<mutex>`), time tracking (`<chrono>`), GUI rendering (`imgui.h`, `gui-context.hpp`), and specific packet layers (Ethernet, IPv4, TCP, Payload).

2. **Performance Metrics Structure:**

   `PerformanceMetrics` keeps track of metrics like latency, packet loss, and bandwidth usage. It also includes methods to record these metrics and detect bandwidth anomalies.

3. **Packet Statistics Structure:**

   `PacketStats` tracks the count of different types of packets (Ethernet, IPv4, TCP, etc.) and has methods to reset these counts, update them when a packet is consumed, and draw the statistics in the GUI.

4. **Packets Data Structure:**

   `PacketsData` holds a list of captured packets and an instance of `PacketStats`.

5. **Packet Capture Callback:**

   `on_packet` is a function that gets called whenever a packet is captured. It processes the packet, updates statistics, records performance metrics, and checks for anomalies.

6. **Device Management:**

   The code retrieves a list of available network devices and allows the user to select one for packet capture. The name of the device is obtained using a macro that handles differences between Windows and other platforms.

7. **Main Function:**

    1. On Windows (`_WIN32`), the entry point is `wWinMain`, while on other platforms, it is `main`.
    2. Initializes logging, GUI context, and retrieves the list of available devices.
    3. The main loop renders the GUI, allowing the user to start and stop packet capture, view packet statistics, and inspect individual packets.

8. **GUI Implementation:**

    1. Uses `ImGui` to create the interface. The interface includes a main menu, a sidebar for packet lists and statistics, and a detailed view for selected packets.
    2. Displays headers and payload information of the selected packet.

4.2 Functional Flow

1. **Initialization:**

    1. Logging and GUI context are initialized.
    2. Available network devices are listed for selection.

2. **Packet Capture:**

    1. When a device is selected, it starts capturing packets using the `on_packet` callback.
    2. Captured packets are processed and stored in `PacketsData`.
    3. Performance metrics are recorded and anomalies are detected during packet processing.

2. **GUI Rendering:**

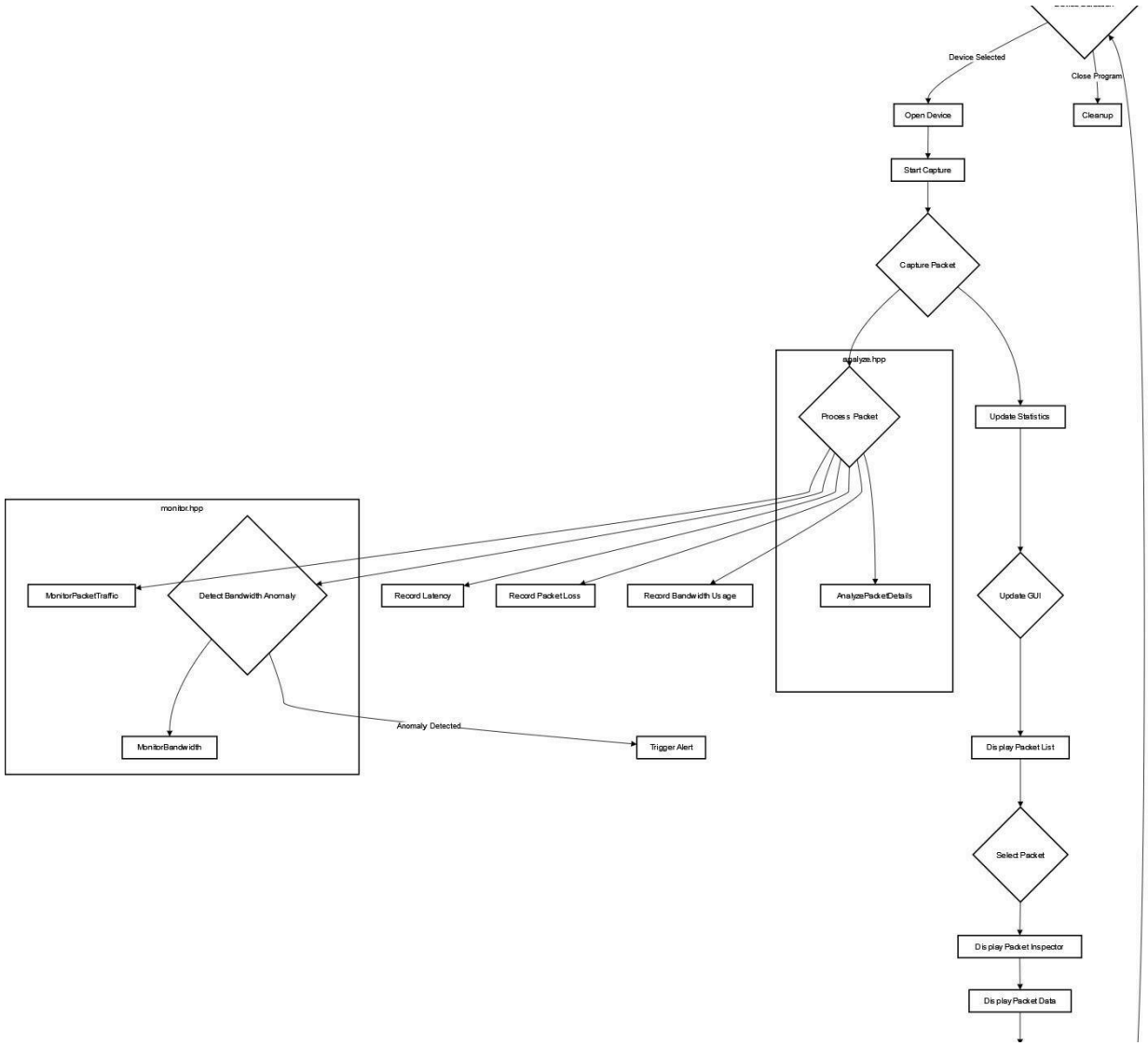
    1. The main menu allows the user to close the application or select a capture device.
    2. The packet list displays counts of different packet types and allows the user to select a packet for detailed inspection.
    3. The details view shows header and payload information of the selected packet.

2. **Metrics and Anomalies:**

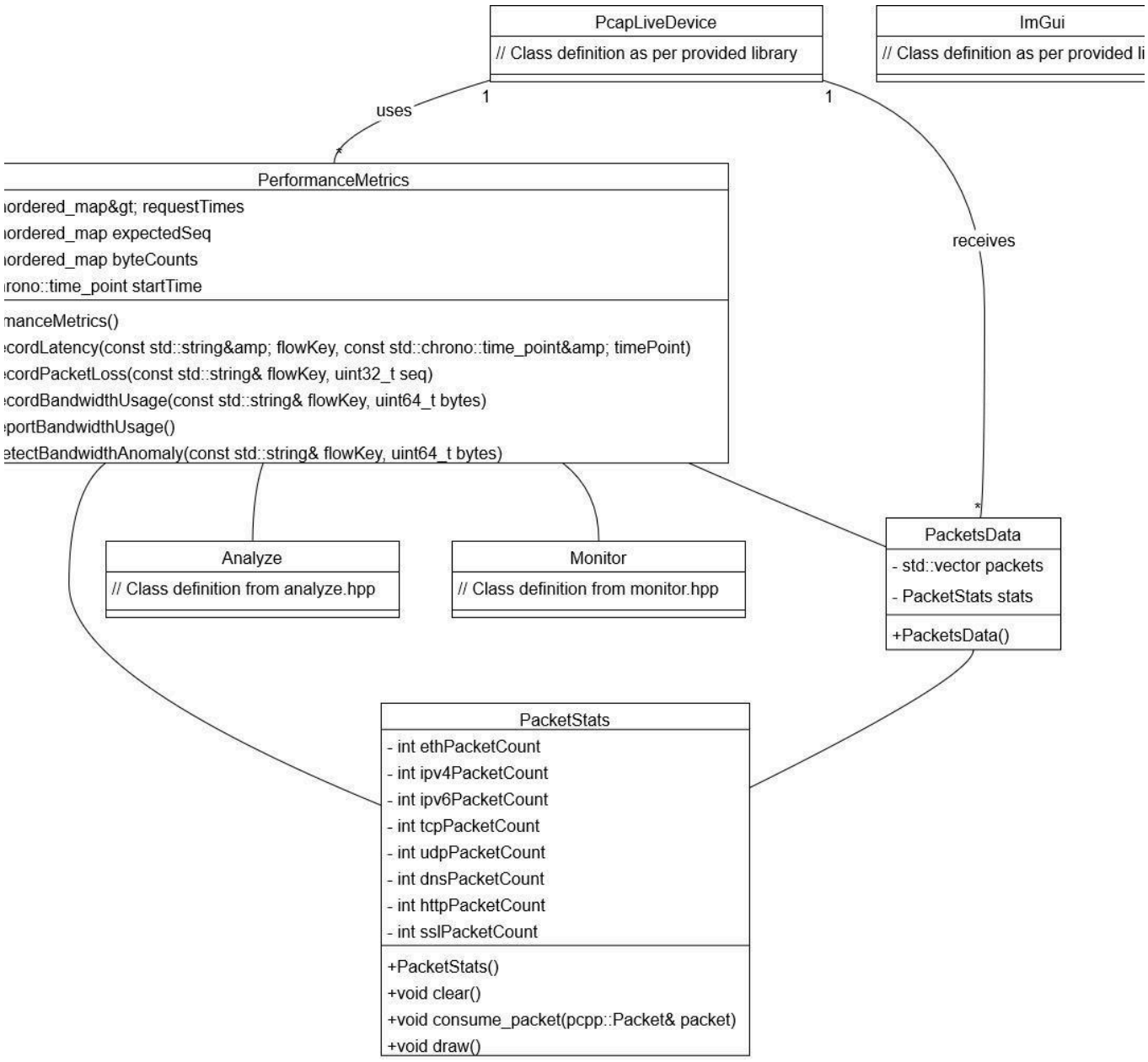
    1. Metrics like latency, packet loss, and bandwidth usage are periodically recorded and reported.
    2. The system checks for unusual bandwidth usage to detect potential network anomalies.

3. **Cleanup:**

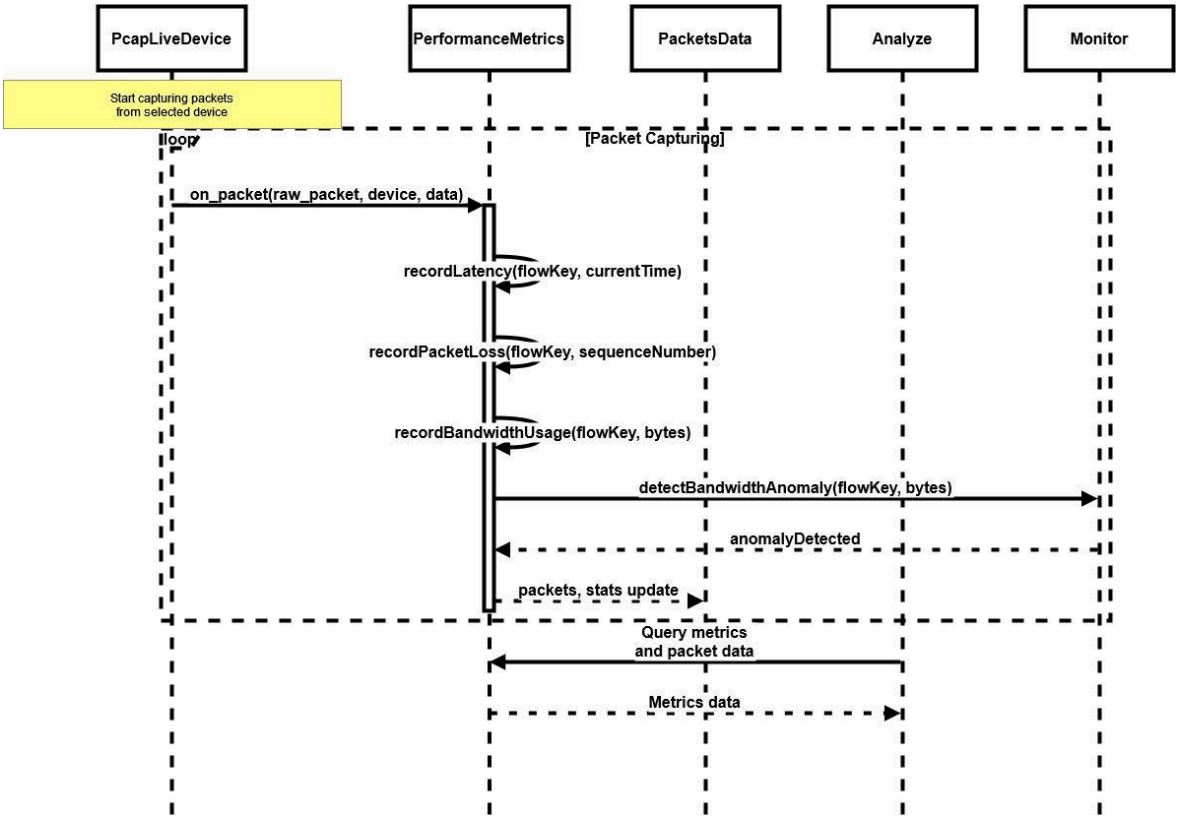
    When the application is closed, any ongoing packet capture is stopped, and the device is properly closed.



Flow diagram of the Packet sniffing



Class Diagram of the Packet sniffing tool



Sequence Diagram (1)

Participants:

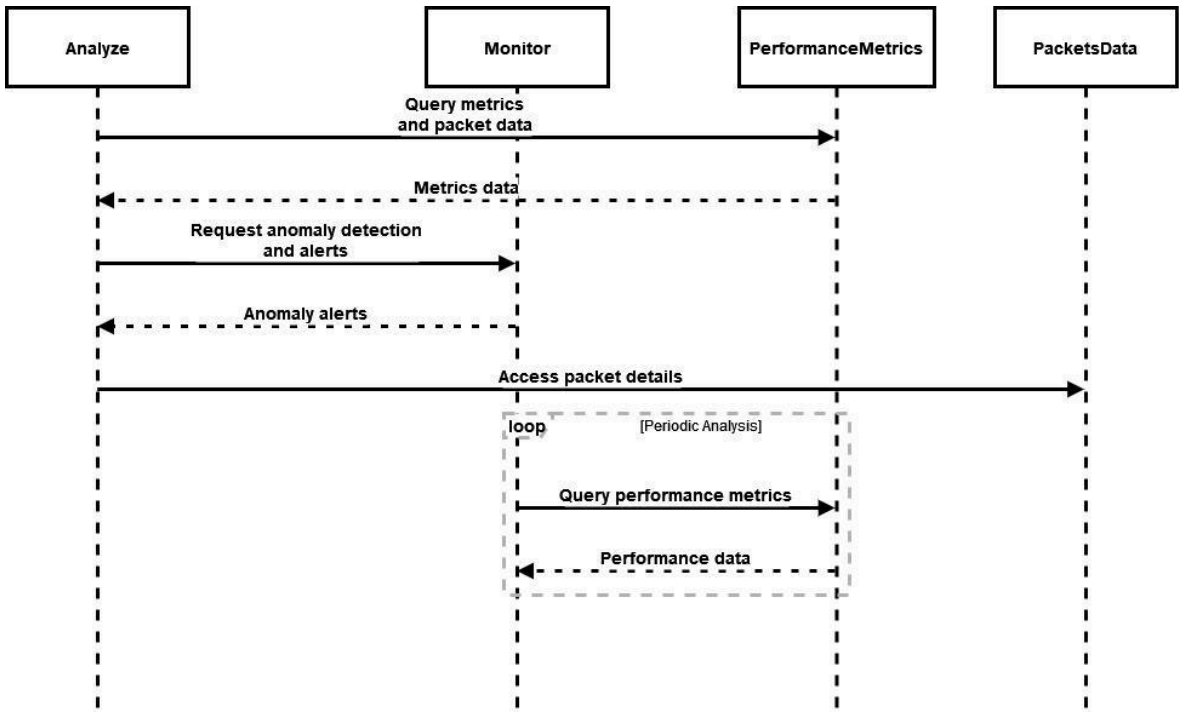
- PcapLiveDevice: Represents the packet capturing device.
- PerformanceMetrics: Manages performance metrics such as latency, packet loss, and bandwidth usage.
- PacketsData: Holds captured packets and statistics.
- Analyze, Monitor: External components interacting with PerformanceMetrics.

Sequence of Events:

- **Packet Capturing Loop**: Illustrates the continuous process of capturing packets. PcapLiveDevice triggers on_packet events, which are handled by PerformanceMetrics.

- **PerformanceMetrics Interactions**: Shows the internal interactions within `PerformanceMetrics` to record latency, packet loss, bandwidth usage, and potential anomalies detected by `Monitor`.

- **External Interaction**: Demonstrates how `Analyze` queries `PerformanceMetrics` for metrics and packet data.



Sequence Diagram (2)

· **Participants**:

- Analyze: Represents the component responsible for querying metrics and packet data.
- Monitor: Manages anomaly detection and alerting.
- PerformanceMetrics: Provides performance metrics data.
- PacketsData: Stores captured packets and statistics.

.

Sequence of Events:

- **Data Query**: Analyze queries PerformanceMetrics for metrics and packet data.
- **Anomaly Detection**: Analyze requests anomaly detection and alerts from Monitor.
- **Periodic Analysis**: Illustrates the periodic querying of performance metrics by Monitor from PerformanceMetrics.



Packet Sniffer

Chapter 5: Implementation and testing

5.1 Technologies and Libraries

1. PcapPlusPlus

PcapPlusPlus is a C++ library that provides a high-level interface for packet capturing, parsing, and crafting. It is built on top of popular packet processing libraries like WinPcap, PcapPlusPluse, and DPDK.
Key Features Used:

- **Packet Capturing**: The library captures packets from live network interfaces or pcap files.
- **Packet Parsing**: PcapPlusPlus can parse various network protocols, such as Ethernet, IPv4, IPv6, TCP, UDP, DNS, HTTP, and SSL, making it easier to analyze the captured traffic.
- **Layer Handling**: It provides classes for different protocol layers (e.g., EthLayer, IPv4Layer, TcpLayer), allowing detailed inspection and manipulation of packets.

2. Dear ImGui

Dear ImGui is a popular, bloat-free graphical user interface library for C++. It is used for creating simple and efficient graphical interfaces, primarily for tools and applications in the gaming and real-time 3D fields.
Key Features Used:

- **Immediate Mode GUI**: Dear ImGui provides an immediate mode graphical user interface, which means the UI is drawn directly from the application code during each frame, simplifying the design and implementation of user interfaces.
- **Widgets and Layouts**: It offers a wide range of widgets and layout options for building complex user interfaces, such as text displays, colored text, and tables.
- **Integration with OpenGL and GLFW**: ImGui can be easily integrated with rendering backends like OpenGL and windowing systems like GLFW, allowing seamless incorporation into various applications.

3. Standard Template Library (STL)

The **Standard Template Library (STL)** is a powerful library in C++ that provides a set of common data structures and algorithms.

Key Features Used:

- **Containers**: The code uses STL containers like std::vector for storing captured packets and std::unordered_map for tracking performance metrics.

- **Algorithms**: STL algorithms are used for operations like searching, sorting, and managing data within containers.

- **Concurrency**: The code uses std::mutex and std::lock_guard for thread-safe operations on shared data structures.

4. Chrono Library

The **Chrono Library** is a part of the C++11 standard library that provides utilities for dealing with time.

Key Features Used:

- **Time Points and Durations**: The code uses std::chrono::time_point and std::chrono::duration to record and calculate time intervals, such as latency and bandwidth usage reporting intervals.
- **High-Resolution Clock**: The high-resolution clock is used for accurate time measurements, essential for performance monitoring and latency calculations.

analyze.hpp
This header file contains functions and definitions for analyzing captured packets. The analysis includes flow identification, latency calculation, and packet loss detection.

Key Functions:

- analyzePacket: Analyzes a captured packet.
- calculateLatency: Computes the latency for different flows.
- detectPacketLoss: Identifies missing packets in a flow.

“monitor.hp”:

This header file contains functions and definitions for monitoring network performance. It includes bandwidth monitoring, anomaly detection, and performance reporting.

Key Functions:

- monitorPacket: Monitors a captured packet.
- reportMetrics: Reports collected performance metrics.
- detectAnomaly: Identifies unusual patterns in network traffic.

Summary

The code combines these technologies and libraries to create a powerful tool for network packet capturing, analysis, and performance monitoring. By leveraging PcapPlusPlus for packet handling, Dear ImGui for GUI rendering, and the STL and Chrono libraries for data management and time calculations, the application provides a comprehensive solution for network traffic analysis and performance monitoring.

5.2 Gui implementation:

5.2.1 GUI Components:

1. **Devices Pane (Top Left)**

- Displays a list of network interfaces available on the system, such as en0, awdl0, llw0, utun0, etc.
- Allows the user to select a specific network interface to monitor packets.

2. **Packets Pane (Top Center)**

- Shows a real-time list of captured network packets.
- Each packet entry contains information such as the protocol layer (e.g., TLS 1.2, TCP, ICMP), packet type (e.g., SYN, ACK, FIN), source port, and destination port.

3. **Packet Inspector (Top Right)**

- Provides detailed information about the selected packet from the "Packets" pane.
- Displays various layers of the packet, such as Ethernet, IPv4, TCP, and application data.
- Shows important packet attributes, such as source and destination MAC and IP addresses, sequence numbers, acknowledgment numbers, flags, etc.

4. **Stats Pane (Bottom Left)**

- Displays statistics about the captured packets, such as the total number of packets, and a breakdown of packets by type (IPv4, IPv6, TCP, UDP,websocket, DNS, HTTP, SSL).

5. **Raw Data Pane (Bottom Right)**

- Displays the raw hexadecimal and ASCII representation of the selected packet's data.
- Allows users to view the raw bytes of the packet for in-depth analysis.

6. **STDOUT Output**

- Shows the command-line output, such as bandwidth usage, latency for specific flow keys, etc.
- Provides real-time feedback and statistics directly from the command-line tool running the packet sniffer.

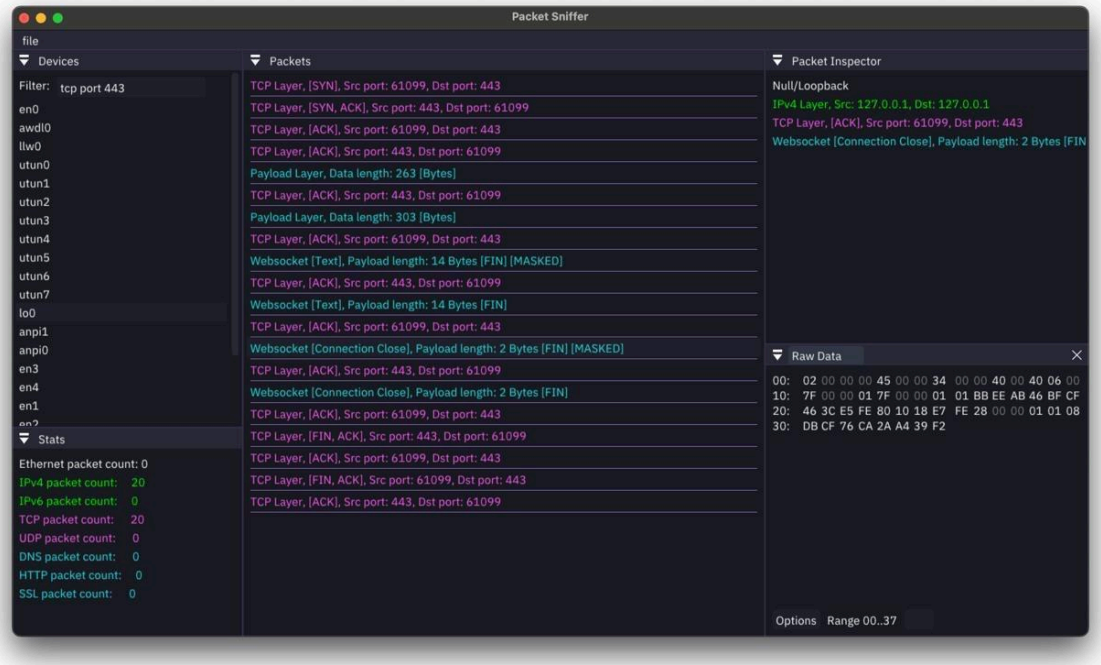
5.3 GUI Design:

Layout: The layout is intuitive, separating the interface into distinct sections for device selection, packet display, detailed inspection, and statistics.

Color Coding: Different colors are used to highlight various protocol layers and packet types, making it easier to distinguish between different types of traffic.

Interactivity: The GUI is interactive, allowing users to filter, select, and inspect packets in real-time.

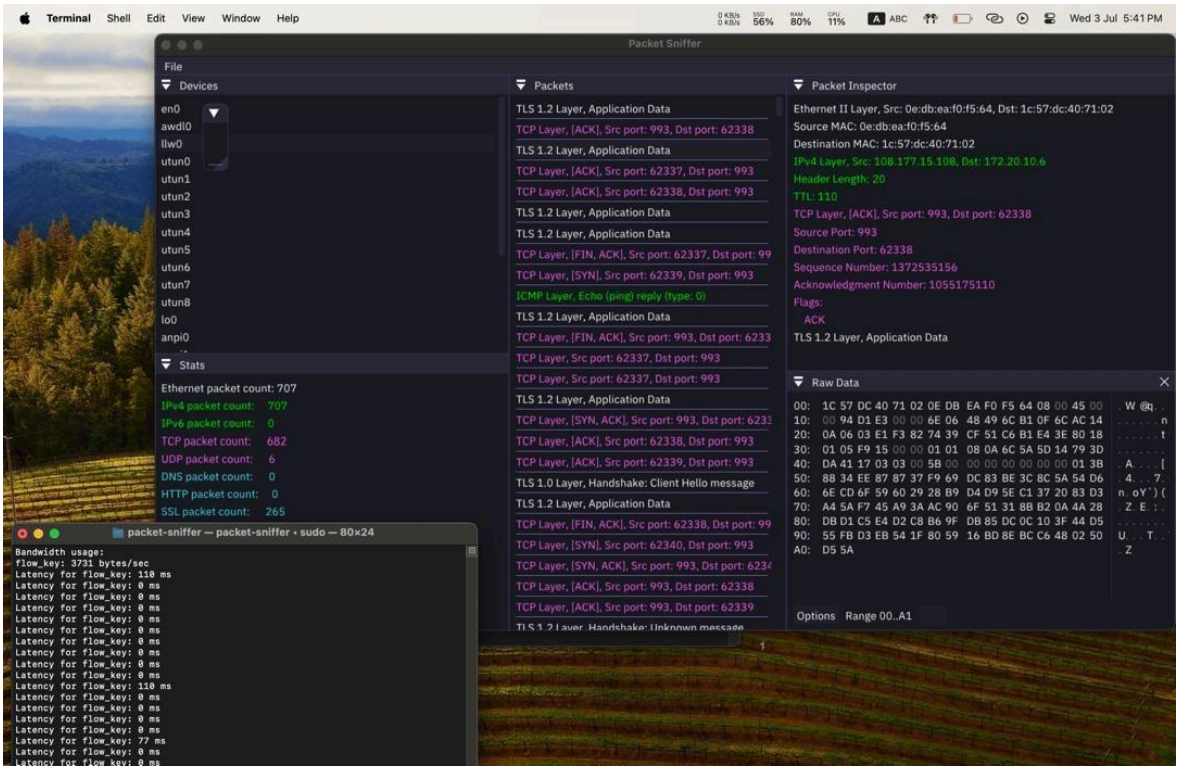
Information Density: The interface is packed with information but organized in a way that prevents it from feeling cluttered. Users can quickly access detailed information without navigating through multiple windows or menus.



Output (1)

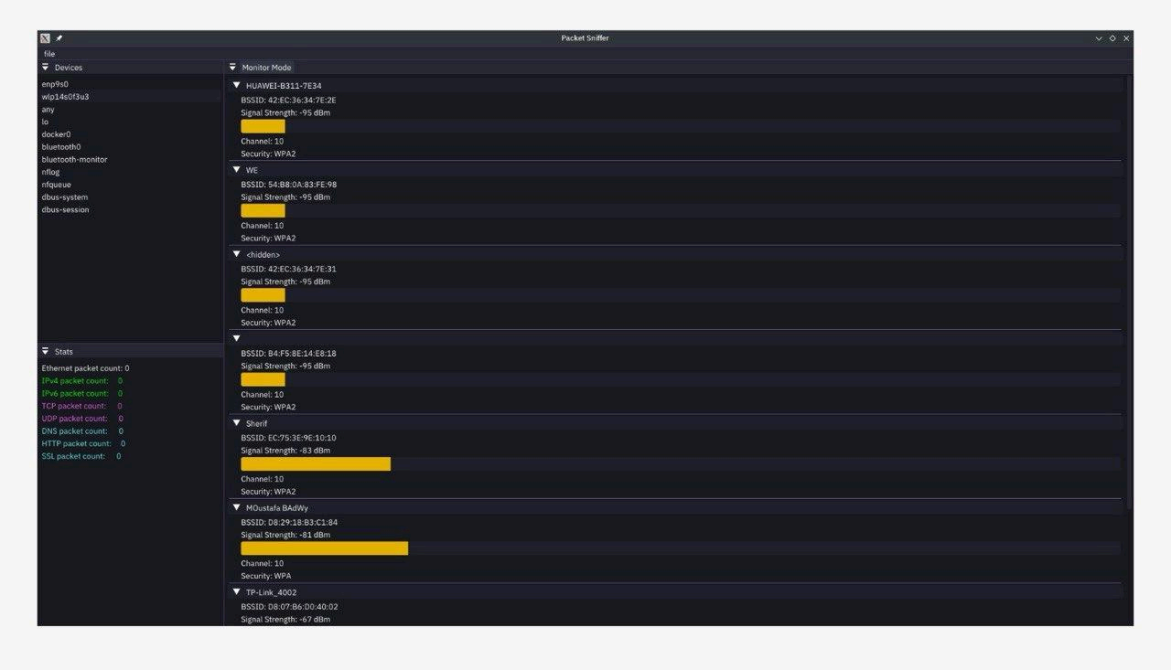
- Display the layout of our tool which consists of devices field that's allow the user to choose which interface they want to capture packets from , stats which display statistics information about the packets ,packets which display the parsed packets.

- Showing the enhanced protocol support by expanding protocol support in PcapPlusPlus to include capturing web-socket packets and showing the headers details in the packet inspection field and the raw data



Output (2)

- Showing extra details about each layer header .
- Print the performance metrics which check for packet loss and bandwidth anomaly , it prints the bandwidth usage every 5 sec .



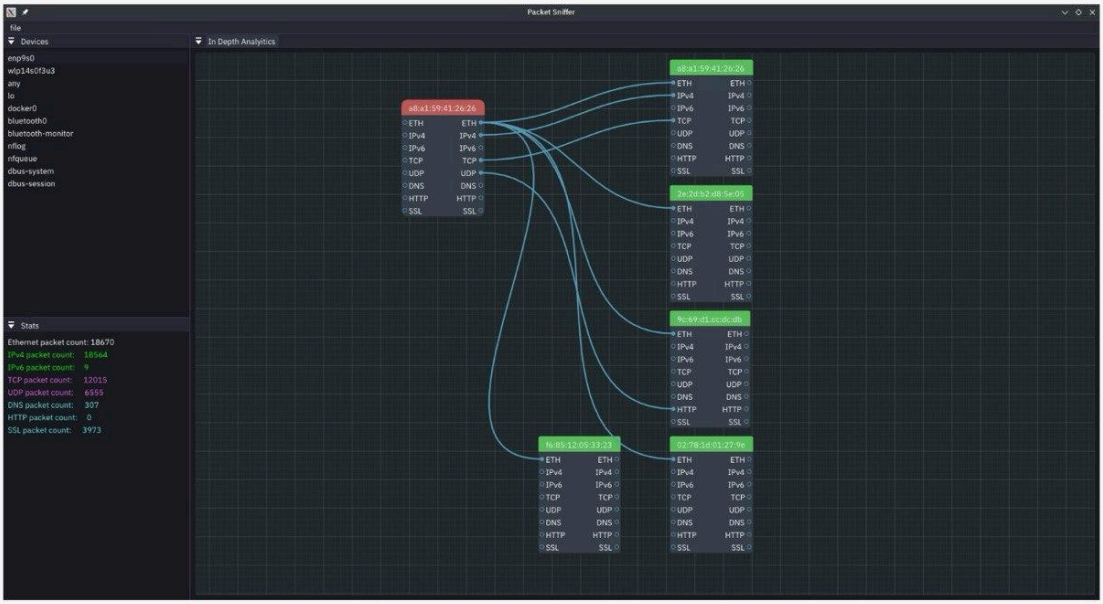
Output (3)

- The third output shows a different mode of the packet sniffer application, likely focused on monitoring wireless networks.
- . Monitor Mode Pane (Top Center)

  - Lists the detected wireless networks in the vicinity.
  - For each network, it shows details such as:
    - Network Name (SSID)
    - BSSID (MAC Address of the Access Point)
    - Signal Strength: Displayed in dBm with a visual bar indicating strength.
    - Channel: The Wi-Fi channel on which the network is operating.
    - Security: The type of security protocol used (e.g., WPA2).

Functionality And Use case:

- Wireless Network Monitoring: This mode is designed for monitoring wireless networks, providing insights into the surrounding Wi-Fi environment. Users can see which networks are available, their signal strengths, and security settings, which can be useful for network troubleshooting and optimization.
- - Network Selection: By listing all available interfaces, the application allows users to choose the best interface for their monitoring needs, whether for wired or wireless networks



Output (4)

- Show Graphical Representation of the connected nodes:
- Graphical Representation:- Nodes:

1. Nodes represent different network entities or devices, identified by their MAC addresses.

- The central node (in red) appears to be a source or central device, while the other nodes (in green) represent connected devices or endpoints.
Each node lists the types of traffic it handles, such as Ethernet (ETH), IPv4, IPv6, TCP, UDP, DNS, HTTP, and SSL.

2. Connections:

- Blue lines represent the communication links between the central node and the other nodes.
- These connections visually indicate the flow of traffic between devices, helping to understand the network topology and interact.



Packet Sniffer

Chapter 6: Conclusion And Recommendations

6.1 Conclusion

In conclusion, the developed application utilizes advanced libraries and technologies to achieve comprehensive network packet capturing, analysis, and performance monitoring capabilities. Leveraging PcapPlusPlus for efficient packet handling and protocol parsing, coupled with Dear ImGui for a user-friendly graphical interface, the application provides real-time insights into network traffic.

6.1.1 Key Achievements

- **Packet Capture and Parsing**: PcapPlusPlus enables the capture and parsing of various network protocols such as Ethernet, IPv4, IPv6, TCP, UDP, DNS, HTTP, and SSL, facilitating detailed packet inspection.
- **Real-time Monitoring**: The application monitors network performance metrics like latency, packet loss, and bandwidth usage using the Chrono library for precise time calculations.
- **Enhanced Protocol Support**: Expand protocol support in PcapPlusPlus to include more specialized or emerging protocols, enabling broader application scenarios and deeper packet analysis.
- **User Interface**: Dear ImGui's immediate mode GUI simplifies the creation of interactive user interfaces, allowing users to visualize and analyze captured packets and performance metrics effectively.

6.2 Recommendations for Future Work

Advanced Analysis Algorithms: Implement advanced algorithms for anomaly detection and pattern recognition in network traffic, improving the application's ability to detect and respond to security threats and performance issues.
Integration with Cloud Services: Explore integration with cloud-based storage and analysis services to store and process captured network data, enabling scalable analysis and long-term storage capabilities.
Graphical Enhancements: Continuously improve the user interface with more intuitive visualizations, customizable dashboards, and real-time charts to enhance user experience and usability.
Performance Optimization: Optimize the application's performance by refining packet processing algorithms, minimizing resource usage, and supporting multi-threaded processing for handling high-volume network traffic efficiently.

6.3 Final Thoughts

The developed application represents a significant advancement in network monitoring and analysis capabilities, offering powerful tools for network administrators, security analysts, and developers alike. By continuing to innovate and expand its capabilities, the application can further solidify its position as a valuable tool in the field of network performance monitoring and cybersecurity.

References

[1] Asrodia, Pallavi, and Hemlata Patel (2012) *analysis of various packet sniffing tools for network monitoring and analysis*, International Journal of Electrical, Electronics and Computer Engineering.

[2] Asrodia, Pallavi, and Hemlata Patel (2012) *Network traffic analysis using packet sniffer*, International Journal of Electrical, Electronics and Com- puter Engineering.

[3] Qadeer, Mohammed Abdul, et al. (2010) *Network traffic analysis and intrusion detection using packet sniffer*, Communication Software and Networks.

[4] Cisco Systems (1992-2006) *Simple Network Management Protocol*, Inter-networking Technologies Handbook, Chpt 56

[5] www.Tcpdump.org

[6] www.wireshark.org

[7] www.ntop.org/guides/pf ring/get started/index.html

[8] www.ntop.org/guides/pf ring/get started/index.html

[9] https://npcap.com/